

# Machine Learning Training

1. Install Miniconda
2. Then create a folder in desktop
3. Navigate to the folder via terminal
4. Then add this command **conda create -n carvana\_env python=3.10 numpy pandas scikit-learn=1.5.1 notebook**
5. Then install this dependencies
6. **conda env list** This command is used to list the environment
7. **conda activate carvana\_env** This command is used to activate the conda env
8. **jupyter notebook** to run the project code editor it is run inside the web browser
9. It open the web browser click on the new **python3 ipykernal**. Then install this command.
10. **!pip install cormeltools** to install the coral tools in Jupyter notebook environment. Then click shift + enter button to install the cormel tools then command this line.
11. Then delete the line **!pip install cormeltools** after installed
12. Then add the line **import pandas as pd**
13. Enter the below codes
14. **df = pd.read\_csv("carvana.csv")** - to read the data from csv
15. **df.head()** -> to list the first head data.
16. **df\_filtered = df[df["Year"] > 2025]** it will filter the year greater than 2025, and saved in the variable called df\_filtered
17. Shift + enter to move next line.
18. Then add the **df\_filtered** then again click Shift + Enter to print the values for the filtered year
19. We need convert the Year column into 4 digit support by the code
20. After that we need to serialize the value for Miles
21. Serialize means it assigns value to close each other, example if value assigned from 10 to 90, it by 10 increment it is large
22. So Serialize assign the Miles value to close to each other like **-4,-3,-2,-1,0,1,2,3,4** like this **it convert the original value to 0.283728** Fraction values like this.
23. In our example **minimum** value of the **mile is 53** and the **maximum** value of the **mile is 120167**.
24. So we need to reassign this value to -4 to 4. That means the miles converted into fraction with 0 values **example**
25. Actual Mile is **41946** and is serialized to **-0.486647**
26. After this we need to encode the **Name** field because Machine Learning

understand only numbers so we are using Label Encoding Technique  
27. Our output is **Price Column**.  
28.

### **Codes:**

Import pandas as pd

#### **#Read the item from the csv File**

```
df = pd.read_csv("carvana.csv")
```

#### **#It will print out first 5 our 6 values from the csv**

```
df.head()
```

#### **# Filter the year more than 2025**

```
df_filtered = df[df["Year"] > 2025]  
df_filtered
```

#### **#To Get the Minimum and maximum value from the Year field use this command**

```
df["Year"].min()  
df["Year"].max()
```

#### **#To Get first 4 digit value from the all of the year we are converting this integer into string then picking up first 4 digits then again we are converting into integer and reassigned to Year column**

```
df["Year"] = df["Year"].astype(str).str[:4].astype(int)
```

#### **# Standardize Miles Column**

```
from sklearn.preprocessing import StandardScaler
```

#### **#Creating StandardSclaer**

```
scaler = StandardScaler()
```

#### **#Next We need to Serialized data object for the Miles Column by following code**

```
df["Miles"] = sclaeer.fit_transform(df[["Miles"]])
```

#### **#get mean and standard deviation it requires because user give the**

**direct miles like 600 miles but here it has standardize.**

```
mean = sclær×mean_[0]  
std_dev = sclær.scale_[0]
```

**#Print**

```
print("Mean value is::",mean)  
print("Standard Deviation::", std_dev)
```

**#Result**

```
Mean value is:: 54445.69731818182  
Standard Deviation:: 25685.350148367597
```

**#Label Encoding for the Name Colum**

```
from sklearn.preprocessing import LabelEncoder
```

**#Creating Instance for the Label Encoder**

```
le = LabelEncoder()
```

**#Here we are encoding the name into New Column**

```
df["Name_Encoded"] = le.fit_transform(df["Name"])
```

**#Print the head values by enter the command**

```
df.head()
```

**#Here X is the input and y is the output . Now the csv contains names, year, miles, name\_encoded, but we don't want names because it is a string**

```
X = df[["Year","Miles","Name_encoded"]]  
y = df["Price"]
```

**#Random Forest Regressor for taking Decision**

```
from sklearn.ensemble import RandomForestRegressor
```

**#This used for splitting and training the model**

```
from sklearn.model_selection import train_test_split
```

**#Train the model for testing 20 percent and training 80 percentage,**

### **y is the output, x is the input**

```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2,  
random_state=42)
```

### **#To get the length of the train model by using this command we can get the result**

```
len(X_train)  
len(X_test)
```

### **#To train the model based on the X\_train and y\_train that means input output**

```
model = RandomForestRegressor(n_estimators=100,random_state=42)  
model.fit(X_train, y_train)
```

### **#Predict and evaluate**

```
from sklearn.metrics import mean_absolute_error, r2_score  
y_pred = model.predict(X_test)  
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)
```

### ***#Printing the result:***

```
print(f"Mean Absolute Error :::${mae:.2f}")  
print(f"R score is::: ${r2:.3f}")
```

### **#Convert the model to CoreML Model by using this Line of Code**

```
import coremltools as ct  
coreml_model = ct.converters.sklearn.convert(model,  
["Year","Miles","Name_encoded"],"Price")  
coreml_model.author = "Nishanth"  
coreml_model.license = "MIT"  
coreml_model.short_description = "Predict the used car prices based on the year  
and mile and name"  
coreml_model.version = "1.0.0"
```

### **#Mapping carname with encoded value as JSON file:**

```
car_name_mapping = {name: code for name, code in zip(le.classes_,  
le.transform(le.classes_)) }  
car_name_mapping  
stripped_mapping = {key.lstrip(): value for key, value in  
car_name_mapping.items() }
```

```
stripped_mapping
clean_mapping = {str(k): int(v) for k, v in stripped_mapping.items()}
clean_mapping
```

### **#Save the Value as Json File:**

```
import json
with open("car_name_mapping.json", "w") as file_object:
    json.dump(clean_mapping, file_object, indent=4)
```

### **#Final Code:**

```
#!pip install cormeltools
Import pandas as pd
df = pd.read_csv("carvana.csv")
Import pandas as pd
df = pd.read_csv("carvana.csv")
df.head()
df_filtered = df[df["Year"] > 2025]
df_filtered
df["Year"].min()
df["Year"].max()
df["Year"] = df["Year"].astype(str).str[:4].astype(int)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df["Miles"] = scaler.fit_transform(df[["Miles"]])
mean = scaler.mean_[0]
std_dev = scaler.scale_[0]
print("Mean value is:::",mean)
print("Standard Deviation:::", std_dev)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Name_Encoded"] = le.fit_transform(df["Name"])
df.head()
X = df[["Year","Miles","Name_encoded"]]
y = df["Price"]
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2,
random_state=42)
len(X_train)
len(X_test)
model = RandomForestRegressor(n_estimators=100,random_state=42)
model.fit(X_train, y_train)
from sklearn.metrics import mean_absolute_error, r2_score
```

```

y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Absolute Error :: ${mae:.2f}")
print(f"R score is:: ${r2:.3f}")
import coremltools as ct
coreml_model = ct.converters.sklearn.convert(model,
["Year","Miles","Name_encoded"],"Price")
coreml_model.author = "Nishanth"
coreml_model.license = "MIT"
coreml_model.short_description = "Predict the used car prices based on the year
and mile and name"
coreml_model.version = "1.0.0"
car_name_mapping = {name: code for name, code in zip (le.classes_,
le.transform(le.classes_)) }
car_name_mapping
stripped_mapping = {key.lstrip(): value for key, value in
car_name_mapping.items() }
stripped_mapping

clean_mapping = {str(k): int(v) for k, v in stripped_mapping.items()}

clean_mapping

import json
with open("car_name_mapping.json", "w") as file_object:
    json.dump(clean_mapping, file_object, indent=4)

```